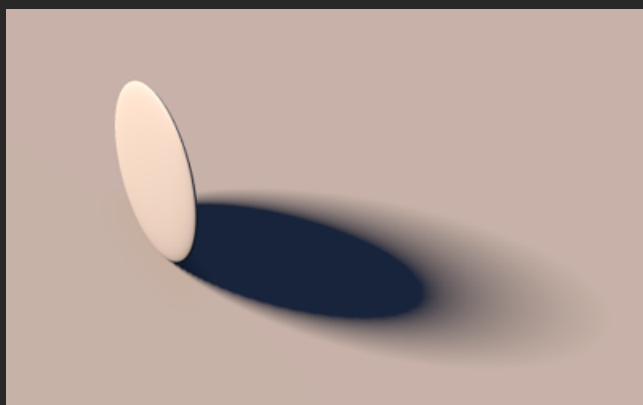


Inigo Quilez :: articles :: ellipsoid SDF - 2008

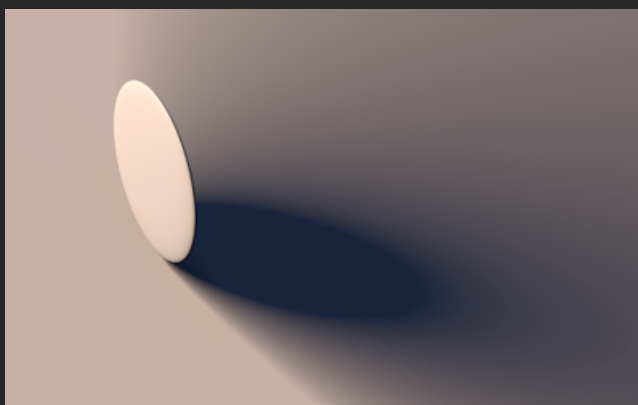


Intro

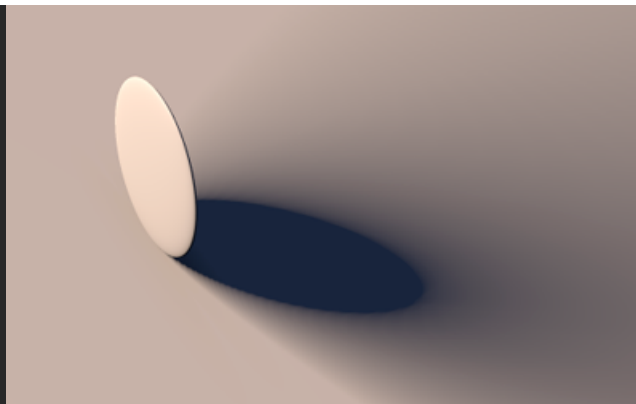
One of the most **useful primitives** when modeling **organic shapes** with SDF is the ellipsoid. However, unlike spheres, cones, boxes and even torii, ellipsoids don't have an analytic distance function that can be evaluated by using roots and/or trigonometric functions. This is bad because that means that in principle one cannot use ellipsoids for modeling, we need to use a numerical method to implement their distance function. That, of course, is pretty slow most (a bisection method would require around 10 to 20 iterations to get good results). So, instead, we need to resort to approximate distance functions. Luckily for us, we CAN find bound functions that at least have a zero iso-surface in the shape of an exact ellipsoid. This means that such bound functions will produce renders of exact ellipsoids, but will report non euclidean distances when queried. That means that a raymarcher will have a harder time finding them and will require more steps until it finds an intersection. It also means occlusion and shadow queries will be wrong for ellipsoids, to different degree based on which bound function we are using. This is an article about two such bound functions.



Shadows produced by an exact SDF



Too large shadows due to a naive bound SDF



Improved shadows with improved bound SDF

Disclaimer

As a matter of fact, it **is** possible to compute the exact distance to an ellipsoid in closed form when the ellipsoid is symmetrical in one axis. In that case, the shape can be evaluated as a 2D ellipse that is revolved along one perpendicular axis in 3D. Since ellipses **do have exact solution** in the form of a cubic, and since creating **shapes of revolution** is trivial and doesn't increase the degree of the polynomials, ellipsoids that are revolved ellipses do have a closed form. All images in this article that compare the two bound techniques use symmetric ellipsoids so I can compare them to the ground truth that I know is exact.

First approach

The simplest approach to bounding the distance to an ellipsoid is stretching the space such that the ellipsoid becomes a sphere, computing the distance to a unit sphere in that space, and then scale the distance back to the original space by the largest of the scale factors. The code would be:

```
float sdbEllipsoidV1( in vec3 p, in vec3 r )
{
    float k1 = length(p/r);
    return (k1-1.0)*min(min(r.x,r.y),r.z);
}
```

Second approach

A simple way to improve the previous bound is to **divide by the length of its gradient**. After some math on paper and nice rearrangement, one gets to this:

```
float sdbEllipsoidV2( in vec3 p, in vec3 r )
{
    float k1 = length(p/r);
    float k2 = length(p/(r*r));
    return k1*(k1-1.0)/k2;
}
```

This involves a second square root (or rather, an inverse-square root, which happens to be faster in that a square root when computed in the GPU). But the extra cost comes with benefits.

Third approach

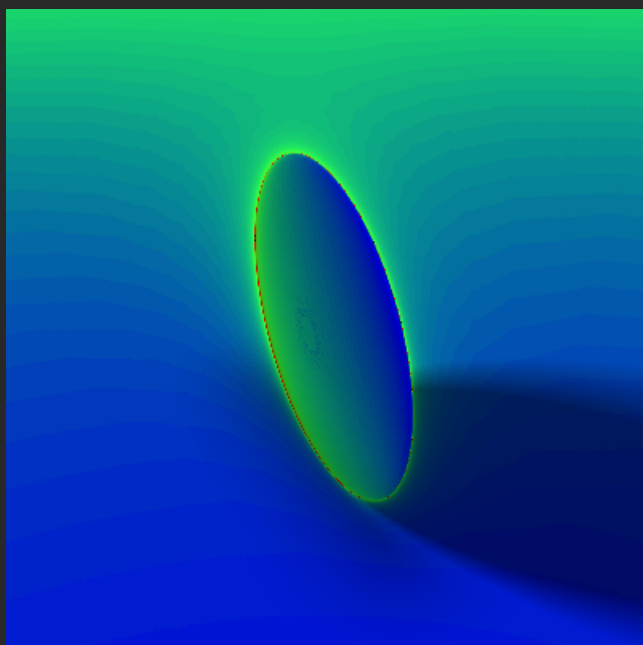
Another idea that might pop in your head is to change the first technique a little bit and see if that improves the distance estimate: stretch the space to deform the ellipsoid into a unit sphere, find the closest point in the unit sphere, transform the point back to the source space, and compute the distance there. The technique actually works great in that **does** produce a better distance estimate. However, it fluctuates between being an underestimate and overestimate, and even with backtracking raymarchers it doesn't seem to work well, and I won't discuss it further in this article, although it can be used for 2D plotting.

```
float sdaEllipsoidV3( in vec3 p, in vec3 r )
{
    float k1 = length(p/r);
    return length(p)*(1.0-1.0/k1);
}
```

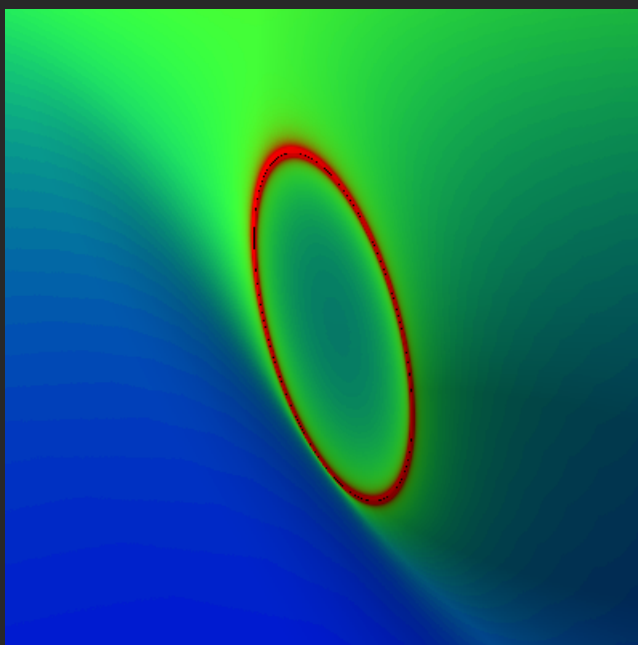
First vs Second techniques

Let's compare the two proposed techniques to the ground truth distance to an ellipsoid in the context of raymarching.

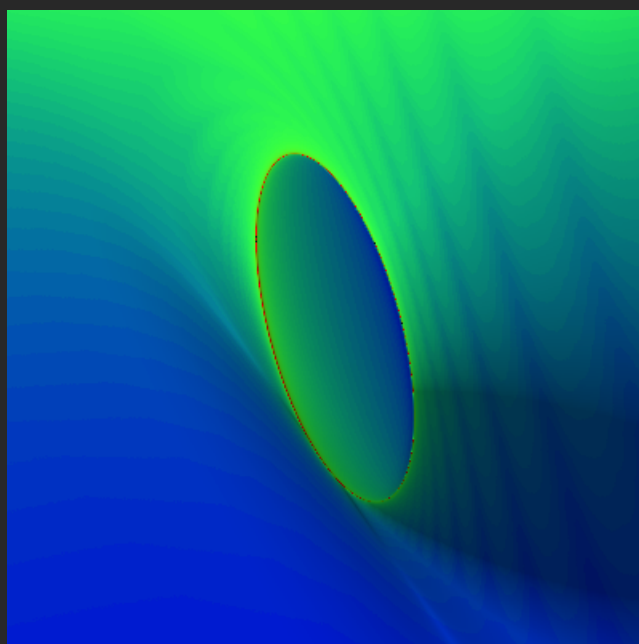
These images below are a map of the number of steps that the raymarcher needed to find the primary ray intersection in the scene rendered at the beginning of the article. Blue means "few steps", around less than 32, green is "moderate number of steps", around 64, and red means "many steps", around 128.



Ground Truth



First Technique



Second 2

As you can see, the First Technique does a poor job at bounding the distance to ellipsoid tightly, while the Second Technique does a much better job, resulting in an image that renders faster, or alternatively, has less artifacts for the same render time. Speaking of that, here goes the comparison of actually fixing the number of

steps and seeing how well the different techniques do at resolving the surface of the ellipsoid for that fixed compute budget:



Ground Truth



First Technique



Second Technique

Indeed, for a set maximum number of iterations of 90, the First Technique is not able to find the surface of the ellipsoid (although it would for 150 iterations), producing black pixels around the edges of the ellipsoid. The Second Technique works much better.

You can find a real-time animated comparison of the two techniques here:

<https://www.shadertoy.com/view/tdS3DG>.

The most important benefit of the Second Technique over First Technique, however, is not the more efficient raymarching. The main benefit is that it produces a much closer distance estimate to the ground truth, and in particular, it makes it much more Euclidean. That means that when combined with other primitives that produce exact Euclidean distances, the Second Technique produces values that play along nicely with those produced by the other primitives. That means that one can adjusting shadow softness parameters, occlusion thresholds and many other values globally for the whole SDF and keep consistent results. With the First Technique the ellipsoids always belong in a different distance, such to speak, universe and are difficult to control.

This is an example of using the Second Technique (left image) vs First Technique (right image) - while both produce the same geometry, the shadows produced by the cap, which is made of two ellipsoid one of which is very flat, are pretty broken and difficult to art direct. The Second Technique produces a sensible distance field and plausible shadows:



Second Technique



First Technique

You can find the real-time animated image and source code that contains the ellipsoid SDF here:

<https://www.shadertoy.com/view/ldd3DX>

inigo quilez - learning computer graphics since 1994